

AUFGABE ZUR VORLESUNG

Dozent Dr. Winkler

Cheat Sheet · Atomar schreiben in Python

Einführung

Ihr Storage in dieser Übung ist ein Verzeichnis mit Dateien. Kein Datenbanksystem steht dazwischen. Damit haben Sie eine Aufgabe, die Ihnen ein DBMS sonst abnimmt: **dafür zu sorgen, dass ein halb ausgeführter Schreibvorgang keinen kaputten Zustand hinterlässt.**

Der Grund ist, dass „eine Datei schreiben“ kein einzelner Vorgang ist. Zwischen Ihrem `write()` und den Bytes auf der Platte liegen drei Stationen, und jede kann Sie überraschen:

```
flowchart LR
    P["Ihr Programm"] -->| "write()" | K["Kernel-Puffer"]
    K -->| "fsync()" | C["Platten-Cache"]
    C -->| "F_FULLFSYNC" (nur auf macOS nötig) | S["Persistent Storage"]

    style S fill:#d4edda
```

Ein zurückgekehrtes `write()` heißt nur, dass der Kernel die Daten hat. Gegen einen Prozessabsturz reicht das; gegen Stromausfall nicht.

Aber nicht nur das - ein einfaches öffnen einer Datei zum schreiben mit `open(pfad, "w")` kürzt die Zieldatei auf null, bevor das erste Byte fließt. Stirbt der Prozess dazwischen, ist der alte Zustand weg - und der neue noch nicht da.

In einem distributed System gibt es oft konkurrierende Schreiber, die zur gleichen Zeit eine Resoruce (Datei) schreiben wollen. Ohne Koordination droht Datenverlust.

Daraus folgen die zwei Fragen, die dieses CheatSheet beantwortet:

1. **Wie mache ich einen Schreibvorgang atomar?** Damit nach einem Absturz entweder der alte oder der neue Zustand dasteht – nie eine Mischung.
2. **Wie koordineiere ich mehrere Schreiber?**

Grundsatz von atomaren Operationen auf Filesystem-Ebene: **Was zusammen gültig werden muss, gehört in eine Datei.** Jede zusätzliche Datei, die mitgepflegt werden muss, ist ein *Dual Write* und damit ein potentielles Problem bei Abstürzen.

(a) Ein Prozess, ein Thread

Gefahr: der **unvollständige Schreibvorgang.** `open(pfad, "w")` kürzt die Zieldatei auf null, bevor das erste Byte fließt. Stirbt der Prozess dazwischen, ist der alte Zustand weg und der neue nicht da.

Das Rezept

✗ So nicht

```
with open(pfad, "w") as f:      # open(..., "w") = O_WRONLY|O_CREAT|O_TRUNC
    os._exit(1)                 # x ABSTURZ – die Datei ist hier schon leer
    json.dump(obj, f)
```

Das `O_TRUNC` im `"w"` kürzt die Zieldatei auf null Bytes, **bevor** das erste Byte des neuen Inhalts fließt. Ein Absturz in dieser Zeile lässt eine leere oder halbe JSON-Datei zurück – der alte Zustand ist weg, der neue nie angekommen.

✓ Musterlösung

```
fd, tmp = tempfile.mkstemp(dir=os.path.dirname(pfad), prefix=".tmp-")
with os.fdopen(fd, "wb") as f:
    f.write(data)
    f.flush()
    os.fsync(f.fileno())        # Daten auf Platte, VOR dem Rename
os.replace(tmp, pfad)           # <- der Commit-Punkt, atomar
fsync_dir(os.path.dirname(pfad)) # den Rename selbst haltbar machen
```

Detail	Warum
Temp-Datei im selben Verzeichnis	<code>os.replace</code> ist nur <i>innerhalb eines Dateisystems</i> atomar. <code>/tmp</code> ist oft ein anderes.
<code>os.replace</code> , nicht <code>os.rename</code>	Gleiche Semantik, aber plattformunabhängig überschreibend.
<code>fsync</code> auf die Datei , vor dem Replace	Der Rename ist sofort sichtbar. Die Daten sind es nicht.
<code>fsync</code> auf das Verzeichnis , nach dem Replace	Sonst kann der Rename selbst den Stromausfall nicht überleben.
<code>.tmp-</code> -Reste beim Start wegräumen	Ein Absturz vor dem Replace lässt sie liegen.

`os.replace` schützt gegen den **Prozessabsturz**. Gegen Stromausfall schützt erst `fsync`, und auf **macOS flusht `fsync` den Plattencache nicht** – dafür braucht es `fcntl.fcntl(fd, fcntl.F_FULLFSYNC)`.

Dies wird oft in einer Hilfsfunktion `write_atomic` encapsulate:

```
import os
import tempfile

def write_atomic(path, data):
    directory = os.path.dirname(path) or "."

    fd, tmp = tempfile.mkstemp(dir=directory)

    try:
        with os.fdopen(fd, "wb") as f:
            f.write(data)
            f.flush()
            os.fsync(f.fileno())

        os.replace(tmp, path)
```

```

except:
    try:
        os.unlink(tmp)
    except FileNotFoundError:
        pass
    raise

```

Datei nur anlegen - falls noch nicht existent

Mit den Flags `O_CREAT` und `O_EXCL` kann man eine Datei anlegen - falls diese nicht schon existiert.

✗ So nicht

```

fd = os.open(pfad, os.O_CREAT | os.O_EXCL | os.O_WRONLY, 0o644)
os._exit(1)          # x ABSTURZ – Name schon belegt, Datei noch leer
os.write(fd, data)
os.close(fd)

```

`O_CREAT | O_EXCL` macht nur den *Anspruch auf den Namen* atomar, nicht den Inhalt. Stirbt der Prozess zwischen `open` und `write`, liegt dort eine **leere Datei** – und jeder Retry liest sie als „schon erledigt, da vorhanden“

✓ Musterlösung

```

fd, tmp = tempfile.mkstemp(dir=d, prefix=".tmp-")
...schreiben, flush, fsync...          # Inhalt fertig, noch namenlos
try:
    os.link(tmp, pfad)                  # <- Commit: scheitert, wenn vergeben
except FileExistsError:
    ...                                # jemand anderes war schneller
os.unlink(tmp)
fsync_dir(d)                           # der neue Name muss haltbar werden

```

`os.link` hängt den Namen an einen Inhalt, der **bereits vollständig auf der Platte liegt**. Entweder der Name existiert mit gültigem Inhalt oder gar nicht; einen Zwischenzustand gibt es nicht.

2. Append-Log – eine Zeile anhängen

✓ Musterlösung

```

with open(pfad, "a") as f:
    f.write(json.dumps(event) + "\n")
    f.flush()
    os.fsync(f.fileno())

```

3. Versionsfeld (die Spalte, die zum `ETag` wird)

✗ So nicht

```

obj = json.loads(open(pfad).read())
# ← x HIER liest der zweite Prozess denselben Stand
obj["version"] = "abc"

```

```
write_atomic(pfad, json.dumps(obj).encode()) # jeder Write gelingt - auch vom
zweiten Prozess
```

Jeder einzelne Schreibvorgang ist sauber, und das Ergebnis ist trotzdem falsch: Zwei Prozesse lesen denselben Stand, beide schreiben, der zweite überschreibt den ersten. **Lost Update**

Mit Hilfe einer Sperre auf einer Datei können sich Prozesse untereinander abstimmen. `fcntl.flock` ist ein Wrapper um den Unix-Systemaufruf `flock(2)`.

✓ Musterlösung

```
def update(pfad, changes):
    lock_path = pfad + ".lock"

    with open(lock_path, "w") as lock:
        fcntl.flock(lock, fcntl.LOCK_EX)

    # Ab hier ist dieser Prozess exklusiv (`LOCK_EX`)
    with open(pfad, "r") as f:
        obj = json.load(f)

    obj.update(changes)
    obj["version"] += 1

    write_atomic(
        pfad,
        json.dumps(obj).encode()
    )

    # beim Verlassen wird der Lock automatisch freigegeben
```

Dateisystem-Operationen in Python — Nachschlageteil

Die Signaturen stammen aus der Python-3.14-Dokumentation.

Schreiben und Committen

Aufruf	Was es tut	Fallstrick
<code>os.replace(src, dst)</code>	benennt um, überschreibt <code>dst</code> still. Atomar, wenn es gelingt.	Nur <i>innerhalb eines Dateisystems</i> . <code>/tmp</code> ist oft ein anderes — Temp-Datei ins Zielverzeichnis . Und: Last-Writer-Wins, überschreibt ohne Rückfrage.
<code>os.rename(src, dst)</code>	dasselbe, aber unter Windows schlägt es fehl, wenn <code>dst</code> existiert	Deshalb immer <code>os.replace</code> nehmen
<code>os.link(src, dst)</code>	zweiter Name für dieselbe Datei. Schlägt mit <code>FileExistsError</code> fehl, wenn <code>dst</code> existiert	First-Writer-Wins — das Gegenteil von <code>os.replace</code> . Taugt als Commit-Punkt, wenn der Inhalt vorher fertig ist
<code>os.write(fd, bytes)</code>	schreibt in den Kernel-Puffer, gibt die Zahl der Bytes zurück	Darf kurz zurückkommen. Rückgabewert prüfen, sonst steht eine halbe Zeile da
<code>os.fsync(fd)</code>	zwingt den Kernel-Puffer auf die Platte	Macht den Inhalt haltbar, nicht den Verzeichniseintrag . Auf macOS er-

Aufruf	Was es tut	Fallstrick
<code>os.sync()</code>	schreibt <i>alles</i> raus	reicht es den Plattencache nicht – dafür <code>fcntl.fcntl(fd, fcntl.F_FULLFSYNC)</code>
<code>f.flush()</code>	leert den Python -Puffer in den Kernel	Nur für Notfälle; blockiert das ganze System
		Ersetzt <code>fsync</code> nicht . Reihenfolge: <code>flush()</code> → <code>fsync(fileno())</code>

Öffnen: die Flags, auf die es ankommt

`os.open(path, flags, mode=0o777)` – Flags mit `|` kombinieren.

Flag	Bedeutung	Wofür hier
<code>O_CREAT O_EXCL</code>	anlegen, aber fehlschlagen , wenn es schon existiert	der UNIQUE -Constraint des Dateisystems
<code>O_APPEND</code>	jeder Write geht ans Dateiende, Positionieren und Schreiben ohne Lücke dazwischen	das Append-only-Log
<code>O_TRUNC</code>	kürzt auf null Bytes	steckt im eingebauten <code>open(p, "w")</code> – die Ursache des zerrissenen Writes
<code>O_WRONLY</code> , <code>O_RDONLY</code> , <code>O_RDWR</code>	Zugriffsart	Pflichtangabe, genau eine davon
<code>O_DIRECTORY</code>	nur Verzeichnisse öffnen	für den Verzeichnis- <code>fsync</code>
<code>O_SYNC</code> , <code>O_DSYNC</code>	jeder Write wartet auf die Platte	korrekt, aber langsam – <code>fsync</code> an der richtigen Stelle ist meist besser

`open(pfad, "w")` ist `O_WRONLY | O_CREAT | O_TRUNC`. Das `O_TRUNC` ist der Grund, warum diese Zeile Ihre alte Datei vernichtet, bevor die neue existiert.

Sperren

Aufruf	Art	Reichweite
<code>fcntl.flock(fd, LOCK_EX)</code>	advisory , hängt am Inode	Prozesse auf einem Host. Über NFS emuliert – und <code>local_lock</code> schaltet es still ab
<code>fcntl.flock(fd, LOCK_EX LOCK_NB)</code>	dasselbe, nicht blockierend	wirft <code>BlockingIOError</code> statt zu warten
<code>fcntl.flock(fd, LOCK_UN)</code>	freigeben	passiert auch beim <code>close()</code> – verlassen Sie sich nicht darauf
<code>fcntl.lockf(fd, LOCK_EX)</code>	POSIX-Record-Lock, bereichsweise	andere Semantik: wird beim Schließen jedes Deskriptors auf die Datei freigegeben
<code>threading.Lock()</code>	prozessintern	nutzlos bei <code>--workers 4</code> – jeder Worker hat sein eigenes

Advisory heißt: Die Sperre wirkt nur zwischen denen, die sie nehmen. Wer direkt schreibt, wird nicht gehindert.

Temporäre Dateien

Aufruf	Anmerkung
<code>tempfile.mkstemp(dir=..., prefix=".tmp-")</code>	legt sicher an, gibt <code>(fd, pfd)</code> zurück. Sie müssen aufräumen
<code>tempfile.mktemp()</code>	deprecated — liefert nur einen Namen, zwischen Name und Anlegen klafft eine Lücke
<code>tempfile.NamedTemporaryFile(delete=False)</code>	Alternative, wenn Sie ein Dateiojekt statt eines Desktors wollen

`dir=` auf das **Zielverzeichnis** setzen — sonst scheitert `os.replace` an der Dateisystemgrenze.

Lesen und Prüfen

Aufruf	Wofür
<code>os.stat(pfad).st_size</code>	Log-Größe als billiger Cache-Validator, O(1)
<code>os.stat(pfad).st_nlink</code>	„Habe <i>ich</i> diesen <code>link()</code> gesetzt?“ — 2 heißt: Temp-Name existiert noch
<code>os.scandir(d)</code>	Verzeichnis lesen, schneller als <code>listdir</code> + <code>stat</code>
<code>pathlib.Path.read_text()</code>	bequem, aber puffernd — für die heiklen Stellen <code>os.read</code> nehmen

Offizielle Dokumentation

Thema	Fundstelle
<code>OS</code> — Betriebssystem-Schnittstellen	https://docs.python.org/3/library/os.html
Dateideskriptor-Operationen	https://docs.python.org/3/library/os.html#file-descriptor-operations
<code>os.open</code> -Flags, vollständige Liste	https://docs.python.org/3/library/os.html#open-constants
<code>fcntl</code> — Sperren (nur Unix)	https://docs.python.org/3/library/fcntl.html
<code>tempfile</code>	https://docs.python.org/3/library/tempfile.html
<code>pathlib</code>	https://docs.python.org/3/library/pathlib.html
<code>shutil</code> — Kopieren, Baumoperationen	https://docs.python.org/3/library/shutil.html

Unterhalb von Python — die Manpages sagen, was wirklich garantiert ist: `man 2 rename`, `man 2 fsync`, `man 2 open`, `man 2 flock`, `man 2 link`. Im Container: `apt-get install manpages-dev`.

Zum Weiterlesen, wenn Sie es genau wissen wollen:

- Pillai et al., *All File Systems Are Not Created Equal* (OSDI 2014) — misst nach, welche Annahmen über atomares Umbenennen auf welchem Dateisystem tatsächlich halten. Ernüchternd.
- Die SQLite-Seite *Atomic Commit In SQLite* (<https://www.sqlite.org/atomiccommit.html>) — das selbe Problem, von Leuten gelöst, die es sehr ernst genommen haben.
- `man 2 fsync` unter **macOS**, Abschnitt zu `F_FULLFSYNC`